

Ответы на вопросы по предмету Обучение с подкреплением

Сычёв Егор 3825М1сФИ1

Оглавление

Неделя 1	2
1. Базовые понятия RL.....	2
2. Метод кросс-энтропии.....	2
Неделя 2	5
1. Функции ценности: $V(s)$ и $Q(s, a)$	5
2. Метод Policy Iteration	6
Неделя 3	7
1. Q-learning и SARSA.....	7
2. Exploration vs Exploitation.....	8
Неделя 4	9
1. DQN.....	9
2. Experience Replay.....	9
3. Target Network.....	10
4. Double DQN.....	11
Неделя 6	12
1. Метод градиента стратегии (Policy Gradient).....	12
2. Алгоритм REINFORCE.....	13
3. Метод Actor-Critic.....	14
Неделя 7	15
1. Применение RNN для генерации последовательностей.....	15
2. Применение RL к дообучению моделей генерации последовательностей.....	16

Неделя 1

1. Базовые понятия RL

Обучение с подкреплением (Reinforcement Learning) — это область машинного обучения, где агент учится принимать решения путем взаимодействия со средой методом проб и ошибок. В отличие от обучения с учителем (Supervised Learning), где алгоритму предоставляются правильные ответы в виде пар «объект-ответ», в RL агент получает лишь **обратную связь** (вознаграждение) за свои собственные действия.

Ключевые отличия от других типов обучения:

- **Обучение с учителем:** Модель аппроксимирует эталонные ответы. Она нуждается в размеченных данных, а действия модели не влияют на будущие входные данные.
- **Обучение без учителя (Unsupervised Learning):** Модель ищет скрытые структуры в данных без какой-либо обратной связи или меток.
- **Обучение с подкреплением:** Агент сам влияет на свои последующие наблюдения через свои действия. Обучение происходит через исследование среды и использование накопленного опыта.

Формализм Марковского процесса принятия решений (MDP)

Для математического описания задач RL используется аппарат **MDP**. Он включает следующие компоненты:

1. **Состояния (States, $s \in S$):** Полное описание ситуации, в которой находится агент.
2. **Действия (Actions, $a \in A$):** Набор возможных шагов, которые агент может предпринять.
3. **Вознаграждения (Rewards, $r \in \mathbb{R}$):** Числовой сигнал, который среда возвращает агенту после совершения действия, отражающий «хорошесть» перехода.
4. **Динамика среды ($P(s_{t+1} | s_t, a_t)$):** Вероятность перехода в новое состояние s_{t+1} при условии текущего состояния s_t и выбранного действия a_t

Марковское предположение: Вероятность перехода в следующее состояние и получение награды зависят только от текущего состояния и действия, а не от всей предыстории:

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} | s_t, a_t)$$

Цель обучения

Целью агента является поиск такой **стратегии (policy, π)**, которая максимизирует суммарное ожидаемое вознаграждение за сессию.

Стратегия $\pi(a | s)$ — это распределение вероятностей выбора действий в данном состоянии: $\pi(a | s) = P(\text{take action } a | \text{in state } s)$.

Суммарная награда (Total Reward, R): $R = \sum_t r_t$

Основная оптимизационная задача формулируется как максимизация математического ожидания награды при следовании стратегии π : $E_{\pi[R]} \rightarrow \max$

2. Метод кросс-энтропии

Метод кросс-энтропии (СЕМ) — это итеративный алгоритм оптимизации стратегии, который относится к категории методов прямого поиска. Общая идея метода заключается в циклическом повторении трех шагов: генерация сессий, отбор лучших сессий и подстройка стратегии под эти сессии.

Общий алгоритм СЕМ:

1. Инициализировать стратегию π (например, случайными весами или равномерным распределением).
2. **Повторять:**
 - Симулировать N сессий, используя текущую стратегию π .
 - Отсортировать сессии по суммарной награде и выбрать M лучших сессий.
 - Обновить параметры стратегии так, чтобы максимизировать вероятность повторения действий, совершенных в элитных сессиях.

Табличный метод кросс-энтропии

Если пространство состояний конечно и невелико, стратегию можно представить в виде матрицы вероятностей $A_{\{s,a\}}$, где строками являются состояния, а столбцами — действия. В этом случае обновление стратегии происходит путем простого подсчета частоты действий в элитных сессиях для каждого состояния: $\pi(a | s) = \frac{(\sum_{\{s_t, a_t \in Elite\} [s_t = s] [a_t = a]})}{(\sum_{\{s_t, a_t \in Elite\} [s_t = s])}$

Если состояние s не встретилось ни в одной элитной сессии, стратегия для него обычно остается равномерной.

Метод кросс-энтропии с использованием нейронной сети

В задачах с огромным или бесконечным пространством состояний табличный подход неприменим. В таких случаях стратегия аппроксимируется параметрической моделью (например, нейронной сетью) $\pi_W(a | s)$

Процесс обучения через максимизацию правдоподобия (Maximum Likelihood): Агент стремится подобрать веса W , которые максимизируют логарифм вероятности выбора «удачных» действий из элитных сессий: $W_{\{i+1\}} = \operatorname{argmax}_W \sum_{\{s_i, a_i \in Elite\}} \log \pi_W(a_i | s_i)$

Для градиентного обновления весов формула: $W_{\{i+1\}} = W_i + \alpha \nabla [\sum_{\{s_i, a_i \in Elite\}} \log \pi_{\{W_i\}}(a_i | s_i)]$

На практике это эквивалентно стандартной задаче классификации (для дискретных действий), где входными данными являются состояния из элитных сессий, а метками классов — соответствующие действия.

Работа с непрерывным пространством действий

Если действия непрерывны, нейронная сеть может предсказывать параметры распределения (например, нормального): $\pi_W(a | s) = N(\mu(s), \sigma^2)$ Здесь $\mu(s)$ — выход нейросети, а σ может быть либо выходом сети, либо обучаемым параметром. Обучение происходит аналогично — через максимизацию логарифма плотности вероятности элитных действий.

Полезные хитрости для стабилизации СЕМ:

- **Хранение прошлых сессий:** Полезно учитывать сессии из последних 3–5 итераций, чтобы обучение шло стабильнее и на большем объеме данных.
- **Регуляризация энтропией:** Добавление штрафа за слишком низкую энтропию стратегии предотвращает преждевременную сходимость к субоптимальному решению.
- **Параллелизация:** Сэмплирование сессий — самая затратная часть, которую легко распределить на несколько ядер CPU или машин.

Неделя 2

1. Функции ценности: $V(s)$ и $Q(s, a)$

Для того чтобы понять, что такое функции ценности, необходимо сначала определить целевую функцию агента. В RL целью является максимизация ожидаемого суммарного вознаграждения, называемого **возвратом (return)**.

Понятие возврата и дисконтирования

Возврат (G_t) — это сумма вознаграждений, полученных агентом начиная с момента времени t до конца эпизода. В задачах с бесконечным горизонтом (где нет четкого конца эпизода) простая сумма может стремиться к бесконечности даже при субоптимальном поведении. Чтобы избежать этого и математически формализовать предпочтение немедленных наград будущим, вводится **коэффициент дисконтирования γ (gamma)**, где $0 \leq \gamma < 1$.

Формула дисконтирования: $G_t \triangleq R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$

Дисконтирование делает сумму конечной.

Функция ценности состояния: $v_{\pi}(s)$ (State-value function)

$v_{\pi}(s)$ — это математическое ожидание возврата, который получит агент, начиная из состояния s и следуя стратегии π в дальнейшем. Интуитивно: «насколько хорошо агенту находиться в данном состоянии».

Определение через математическое ожидание: $v_{\pi}(s) \triangleq E_{\pi}[G_t | S_t = s]$

уравнение ожидания Беллмана для v_{π} : $E v_{\pi}(s) = \sum_a \pi(a|s) \sum_{\{r, s'\}} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]]$

Здесь ценность состояния зависит от:

1. Вероятности выбора действия согласно стратегии $\pi(a|s)$.
2. Динамики среды $p(r, s' | s, a)$.
3. Немедленной награды r и дисконтированной ценности следующего состояния $v_{\pi}(s')$.

Функция ценности действия: $q_{\pi}(s, a)$ (Action-value function)

$q_{\pi}(s, a)$ — это ожидаемый возврат, если агент находится в состоянии s , совершает конкретное действие a , а затем следует стратегии π . Интуитивно: «насколько выгодно совершить данное действие прямо сейчас».

Определение: $q_{\pi}(s, a) \triangleq E_{\pi}[G_t | S_t = s, A_t = a]$

Связь между $v_{\pi}(s)$ и $q_{\pi}(s, a)$: Ценность состояния — это среднее значение ценностей всех возможных действий в нем, взвешенное по вероятности их выбора: $v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a)$

q_{π} можно выразить через v_{π} следующего состояния: $q_{\pi}(s, a) = \sum_{\{r, s'\}} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]]$

2. Метод Policy Iteration

Главная цель решения MDP — найти **оптимальную стратегию π^*** , которая максимизирует ожидаемый возврат во всех состояниях. Метод **Policy Iteration** реализует концепцию **обобщенной итерации стратегии (Generalized Policy Iteration, GPI)**, которая состоит из чередования двух процессов: оценки и улучшения.

Этап 1: Оценка стратегии (Policy Evaluation)

Задача — вычислить функцию ценности v_π для текущей стратегии π . Уравнение Беллмана для v_π представляет собой систему линейных уравнений, где количество неизвестных равно количеству состояний. Вместо прямого решения системы часто используется итерационный подход: $V(s) \leftarrow \sum_a \pi(a|s) \sum_{\{s', r\}} p(s', r | s, a) [r + \gamma V(s')]$ Процесс повторяется до тех пор, пока изменения Δ не станут меньше порога θ .

Этап 2: Улучшение стратегии (Policy Improvement)

Зная ценность состояний v_π для текущей стратегии, мы можем попытаться сделать её лучше. Самый простой способ — действовать **жадно (greedy)** относительно вычисленной функции ценности. Мы создаем новую стратегию π' , выбирая действие, максимизирующее $q_{\pi(s, a)}$: $\pi'(s) \leftarrow \arg \max_a \sum_{\{r, s'\}} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$

Теорема об улучшении стратегии гарантирует, что новая стратегия π' будет не хуже старой π ($v_{\pi'}(s) \geq v_\pi(s)$ для всех s). Если π' совпадает с π , то согласно уравнениям оптимальности Беллмана, такая стратегия является оптимальной.

Полный алгоритм Policy Iteration:

1. **Инициализация:** Произвольные $V(s) \in \mathbb{R}$ и $\pi(s) \in A$ для всех состояний.
2. **Policy Evaluation:** Итеративное вычисление $V \approx v_\pi$ до сходимости.
3. **Policy Improvement:**
 - Для каждого состояния s : запомнить $\text{old-action} = \pi(s)$.
 - Обновить $\pi(s)$ через $\arg \max$ по $q_{\pi(s, a)}$.
 - Если хотя бы в одном состоянии действие изменилось ($\text{old-action} \neq \pi(s)$), вернуться к шагу 2.
 - Если стратегия стабильна — вернуть $v^* \approx V$ и $\pi^* \approx \pi$.

Сравнение с Value Iteration

В отличие от **Value Iteration**, где оценка стратегии обрывается уже после одной итерации обновления $V(s)$, **Policy Iteration** проводит полную оценку текущей стратегии до сходимости на каждом этапе.

- **PI** требует меньше итераций для поиска оптимальной стратегии, но каждая итерация (оценка) обходится дороже.
- **VI** быстрее на каждой отдельной итерации, но их требуется гораздо больше.

Неделя 3

1. Q-learning и SARSA

В реальных задачах мы редко обладаем полной информацией о динамике среды (вероятностях переходов $P(s', r | s, a)$). Это заставляет нас переходить от динамического программирования к методам **Model-free**, где агент обучается на основе реального или симулированного опыта — **траекторий** (последовательностей состояний, действий и наград).

Переход от $V(s)$ к $Q(s, a)$

функция ценности состояния $V(s)$ становится бесполезной, если мы не знаем $P(s'|s, a)$, поэтому в обучении без модели основной целью становится изучение **функции ценности действия $Q(s, a)$** — ожидаемого возврата при совершении действия a в состоянии s .

Идея временных различий

Оба алгоритма (Q-learning и SARSA) базируются на идее **TD-обучения**. Вместо того чтобы ждать конца эпизода, как в методах Монте-Карло, агент обновляет свои оценки на каждом шаге, используя текущую награду и оценку ценности следующего состояния. Общая формула обновления выглядит так: $Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha [r_t + V(s_{t+1}) - V(s_t)]$, где $Q(s_t, a_t)$ — это «улучшенная» оценка, полученная из уравнения Беллмана.

Алгоритм Q-learning (Off-policy)

Q-learning — это метод обучения **вне стратегии (off-policy)**. Это означает, что агент учится оптимальному поведению независимо от того, какие действия он совершает в процессе исследования. Для обновления $Q(s, a)$ алгоритм использует «жадную» оценку следующего состояния, предполагая, что в будущем он выберет наилучшее возможное действие:
$$Q(s, a) = r(s, a) + \gamma \max_{a_i} Q(s', a_i)$$

Цикл алгоритма:

1. Инициализировать $Q(s, a)$ нулями.
2. Получить переход $\langle s, a, r, s' \rangle$ из среды.
3. Вычислить целевое значение $\hat{Q}$ через $\max$ по следующим действиям.
4. Обновить таблицу Q .

Алгоритм SARSA (On-policy)

Название расшифровывается как **State-Action-Reward-State-Action**. Это метод обучения **по стратегии (on-policy)**, то есть агент оценивает именно ту стратегию, которой он следует в данный момент (включая все случайные исследовательские действия). Обновление происходит по формуле: $Q(s, a) = r(s, a) + \gamma Q(s', a')$. Здесь a' — это действие, которое агент **действительно выбрал** в следующем состоянии s' , следуя своей текущей стратегией.

2. Exploration vs Exploitation

Одной из проблем RL, является баланс между использованием уже накопленных знаний (**exploitation**) и поиском новых, потенциально лучших решений (**exploration**).

Почему нельзя всегда выбирать argmax ?

Если агент всегда будет выбирать действие с максимальным текущим $Q(s, a)$, он может попасть в ловушку локального оптимума. В лекции приводится пример робота, который учится ходить: если он случайно нашел способ ползать и это дает небольшую положительную награду, без исследования он никогда не рискнет встать на ноги, так как первые попытки ходить принесут падения и отрицательные награды.

Стратегия ϵ -greedy

Это самый простой и распространенный метод балансировки:

- С вероятностью ϵ агент выбирает абсолютно случайное действие (**exploration**).
- С вероятностью $1 - \epsilon$ агент выбирает лучшее действие согласно текущим знаниям (**exploitation**).

Для обеспечения сходимости к оптимальной стратегии применяется **Exploration over time**: параметр ϵ постепенно уменьшается (например, с 0.5 до 0.01). Если $\epsilon \rightarrow 0$, то стратегия становится «жадной в пределе».

Стратегия Softmax

ϵ -greedy плох тем, что он выбирает все «не-лучшие» действия с равной вероятностью.

Метод **Softmax** выбирает действия пропорционально их ценности: $\pi(a | S) = \frac{\exp(\frac{Q(s,a)}{\tau})}{\sum_{\{a_i\}} \exp(\frac{Q(s,a_i)}{\tau})}$, где

τ (**tau**) — температура. При высокой температуре действия выбираются почти равномерно, при низкой — агент склоняется к самому выгодному.

Experience Replay и Off-policy обучение

Важным следствием разделения стратегий является возможность использования **Experience Replay**. Поскольку Q-learning является **off-policy**, мы можем сохранять старые переходы $\langle s, a, r, s' \rangle$ в буфер и обучаться на них многократно. Это позволяет:

1. Не посещать одно и то же состояние слишком часто для обучения.
2. Использовать данные, полученные от «старых» или более слабых версий стратегии. Однако этот метод **не работает** для SARSA, так как SARSA требует данных, полученных строго от текущей политики.

Алгоритмы N-step

рассматриваются **N-step** варианты алгоритмов занимают промежуточное положение между TD и Монте-Карло. Они используют награды за n шагов перед тем, как добавить оценку будущего состояния. Формула для N-step SARSA: $Q(s_t, a_t) = [\sum_{\tau=t}^{t+n-1} \gamma^{t-\tau} r(s_\tau, a_\tau)] + \gamma^n Q(s_{t+n}, a_{t+n})$. Такие методы позволяют информации о награде распространяться назад по цепочке состояний гораздо быстрее.

Неделя 4

1. DQN

Проблема размерности.

Классический табличный Q-learning предполагает хранение значений $Q(s,a)$ для каждой пары состояние-действие. Однако в реальных задачах пространство состояний огромно или непрерывно. Если входными данными являются изображения (210x160x3 пикселей), количество возможных комбинаций делает использование таблиц невозможным.

Идея аппроксимации.

Вместо хранения таблицы мы используем параметрическую функцию (нейронную сеть) с весами W , которая предсказывает значения Q : $Q(s, a, W) \approx Q^*(s, a)$. В архитектуре DQN сеть обычно принимает на вход состояние s (кадр игры) и за один проход выдает значения Q для всех возможных действий в этом состоянии. Это вычислительно эффективнее, чем подавать пару (s,a) отдельно для каждого действия.

Обучение как задача регрессии.

Обучение DQN сводится к минимизации среднеквадратичной ошибки (MSE) между текущим предсказанием сети и целевым значением (Target), полученным из уравнения оптимальности Беллмана (Critic).

Целевое значение: $Q_{target} = r + \gamma Q'(s_{next}, \mu'(s_{next}))$

Функция потерь: $L_{Critic} = (Q(s,a) - Q_{target})^2$ и $L_{Actor} = -Q(s, \mu(s))$

Важным аспектом является то, что при вычислении градиента целевое значение рассматривается как константа (Stop gradient), хотя оно и зависит от тех же весов W . Без этого обучение становится крайне нестабильным.

2. Experience Replay

Проблема корреляции данных.

В стандартном обучении нейросетей предполагается, что обучающие примеры являются независимыми и одинаково распределенными (i.i.d.). В RL это условие нарушается: последовательные кадры в игре (траектория агента) сильно коррелируют между собой. Кроме того, если агент долго находится в одной локации (например, «под левым камнем»), сеть быстро забывает, как вести себя в других частях среды, что приводит к деградации кривой обучения.

Механизм Replay Buffer.

Чтобы разорвать временные связи, предлагается сохранять прошлые взаимодействия агента со средой в виде кортежей $\langle s, a, r, s' \rangle$ в специальную память — **Replay Buffer**. В процессе обучения:

1. Агент совершает шаг в среде и записывает переход в буфер.
2. Для обновления весов сети из буфера случайно сэмпляется батч (batch) данных из разных эпизодов и моментов времени.

Преимущества:

- Данные становятся ближе к распределению i.i.d..
- Опыт используется многократно, что повышает эффективность обучения.
- Это позволяет реализовать **Off-policy** обучение: агент учится на данных, полученных «прошлой версией себя» (когда стратегия была слабее). Стоит отметить, что методы вроде SARSA или CEM могут «сломаться» при использовании Experience Replay, так как они критически зависят от текущей стратегии (on-policy).

3. Target Network

Проблема автокорреляции целей.

В базовом DQN одна и та же сеть используется и для предсказания текущего значения $Q(s, a)$, и для вычисления целевого значения $r + \gamma \max_{a'} Q(s', a')$. Это создает эффект «погони за собственным хвостом»: когда мы обновляем веса, чтобы приблизить $Q(s, a)$ к цели, сама цель тоже сдвигается, так как веса обновились. Это часто приводит к расходимости алгоритма.

Решение с замороженными весами.

Вводится вторая сеть — **Target Network** с параметрами Θ^- . Она используется исключительно для вычисления целевого значения:

$$L(\Theta) = E_{\{s \sim S, a \sim A\}} [(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta^-)))^2]$$

Параметры целевой сети Θ остаются фиксированными (замороженными) на протяжении многих шагов обучения.

Методы обновления Θ^- :

1. **Hard Update:** Каждые n шагов (например, каждые 5000 шагов в Atari) веса целевой сети полностью копируются из основной сети: $\Theta^- \leftarrow \Theta$.
2. **Soft Update:** Веса обновляются постепенно на каждом шаге:
$$\Theta^- = (1 - \alpha)\Theta^- + \alpha\Theta$$
3. Где α — очень малое число (например, 0.001). Это обеспечивает максимальную стабильность целей.

4. Double DQN

Проблема переоценки.

Использование оператора $\max$ в целевом значении Беллмана приводит к систематическому завышению оценок Q -значений. Это происходит из-за того, что даже если истинные ценности действий равны, случайный шум в аппроксимации может сделать одно действие чуть «лучше» других, и именно оно будет выбрано оператором $\max$. Математически это выражается неравенством: $E[\max_a Q(s', a')] \geq \max_a E[Q(s', a')]$. В экспериментах (например, в рулетке) DQN может показывать огромную ожидаемую прибыль там, где она на самом деле отрицательна.

Идея Double Q-learning.

Для решения проблемы необходимо разделить процесс **выбора** лучшего действия и процесс **оценки** его стоимости. В Double DQN это реализуется через использование двух сетей (основной и целевой):

1. Основная сеть Θ выбирает действие, которое она считает лучшим:
 $a^* = \operatorname{argmax}_a Q(s', a, \Theta)$
2. Целевая сеть Θ^- оценивает стоимость этого выбранного действия.

Итоговая формула Double DQN Target:

$$y = r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a', \Theta), \Theta^-)$$

Такой подход значительно снижает смещение оценок и позволяет агенту сходиться к более стабильным и реалистичным стратегиям. Это улучшение стало одним из ключевых компонентов финального алгоритма **Rainbow**.

Неделя 6

1. Метод градиента стратегии (Policy Gradient)

Ранее рассматривались методы, основанные на ценности (Value-based), такие как Q-learning, где агент сначала учит функцию ценности, а затем выводит стратегию через $a = \operatorname{argmax} Q(s, a)$. Однако, вычисление точных Q-значений зачастую оказывается более сложной задачей, чем непосредственный выбор оптимального действия.

Основные идеи:

- **Прямое обучение:** Мы обучаем параметрическую модель стратегии $\pi_\theta(a|s)$ напрямую, без явного вычисления функции ценности для выбора действий.
- **Стохастичность:** В отличие от жадных стратегий, методы градиента стратегии обычно используют стохастические политики. Это позволяет естественным образом решать проблему исследования (exploration), так как процесс сэмплирования действий сам по себе обеспечивает разнообразие опыта.
- **Непрерывные пространства:** Одним из главных преимуществ является поддержка непрерывных пространств действий (например, управление напряжением мотора или распределение весов в портфеле), где выбор argmax по бесконечному числу действий невозможен.

Математическое обоснование

Целью обучения является максимизация ожидаемого вознаграждения $J = E_{\pi_\theta}[R(s, a)]$. Главная трудность заключается в том, что параметры θ находятся внутри оператора математического ожидания (под распределением π_θ). Чтобы вычислить градиент $\nabla_\theta J$, используется «трюк с логарифмической производной»:

$$\nabla \log \pi(z) = \left(\frac{1}{\pi(z)} \right) \nabla \pi(z) \Rightarrow \nabla \pi(z) = \pi(z) \nabla \log \pi(z)$$

Применяя это к функционалу ожидаемой награды, мы получаем **аналитический градиент**: $\nabla_\theta J = \int_s p(s) \int_a \nabla_\theta \pi_\theta(a|s) R(s, a) da ds = E_{\{s, a\}}[\nabla_\theta \log \pi_\theta(a|s) R(s, a)]$

Это выражение позволяет нам оценивать градиент с помощью сэмплирования обычных сессий, не зная динамики среды.

2. Алгоритм REINFORCE

Алгоритм **REINFORCE** является классической реализацией метода градиента стратегии, основанной на методе Монте-Карло.

Механика алгоритма:

1. Инициализируются веса нейронной сети θ_0 .
2. В цикле генерируется N полных игровых сессий согласно текущей стратегии π_θ .
3. Для каждого шага в сессии вычисляется оценка градиента. В базовом случае (бандит) используется награда $R(s,a)$, а в дисконтированном случае — возврат
$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$
4. Параметры обновляются по принципу градиентного подъема: $\theta_{\{i+1\}} \leftarrow \theta_i + \alpha \nabla_{\theta} J$

Особенности и ограничения:

- **On-policy:** REINFORCE требует, чтобы данные были получены строго по текущей стратегии. Мы не можем использовать старый опыт (Experience Replay), так как градиент логарифма вероятности привязан к конкретному распределению.
- **Высокая дисперсия:** Поскольку оценка градиента зависит от полной суммы наград G_t , которая может сильно варьироваться от сессии к сессии, обучение часто бывает нестабильным.

Метод Baseline

Для снижения дисперсии из оценки возврата вычитают некоторую величину $b(s)$, которая не зависит от действия a . Математически доказано, что это не меняет направление градиента в среднем, так как $E[\nabla \log \pi_\theta(a|s) b(s)] = 0$, но значительно уменьшает разброс значений. Наилучшим выбором для $b(s)$ является функция ценности состояния $V(s)$.

3. Метод Actor-Critic

Метод **Actor-Critic** — это гибридный подход, объединяющий преимущества методов на основе ценности и градиента стратегии.

Архитектура:

- **Actor:** Обучает стратегию $\pi_{\theta}(a|s)$ (как в REINFORCE), отвечая за выбор действий.
- **Critic:** Обучает функцию ценности $V_{\theta}(s)$ (как в методах на основе ценности), оценивая «хорошесть» состояний.

Advantage Actor-Critic

Вместо полной награды G_t Актер использует функцию преимущества (Advantage) $A(s,a)=Q(s,a)-V(s)$. Это позволяет понять, насколько выбранное действие лучше среднего действия в этом состоянии. Чтобы не учить отдельно Q и V , часто используют TD-ошибку: $A(s, a) = r + \gamma V(s') - V(s)$

Функции потерь для обновления весов:

1. **Для Актера:** Максимизация

$$\nabla J_{actor} \approx \left(\frac{1}{N}\right) \sum_{\{i=0\}}^N \sum_{\{s,a \in z_i\}} \log \pi_{\theta(a|s)} \cdot A(s, a)$$

2. **Для Критика:** Минимизация MSE между предсказанным $V(s)$ и целевым значением $r + \gamma V(s')$

$$L_{critic} \approx \left(\frac{1}{N}\right) \sum_{\{i=0\}} \sum_{\{s,a \in z_i\}} (V_{\theta(s)} - [r + \gamma V(s')])^2$$

Практические советы:

- **Регуляризация энтропией:** К лоссу Актера добавляют энтропию стратегии, чтобы предотвратить преждевременную сходимость и заставить агента продолжать исследование.
- **Параллелизм:** Асинхронное обучение на нескольких копиях среды (Workers) позволяет собирать более разнообразные данные без использования Experience Replay.

Неделя 7

1. Применение RNN для генерации последовательностей

Рекуррентные нейронные сети являются фундаментальным инструментом для работы с последовательными данными, такими как временные ряды, аудиосигналы и тексты. В отличие от стандартных сетей прямого распространения, RNN обладают «памятью», позволяющей им учитывать контекст всей предыстории при принятии решений на текущем шаге.

Моделирование языка

Основная задача генерации последовательностей в контексте текста — это построение **языковой модели**. Цель модели состоит в изучении распределения вероятностей текста $P(text)$, которое можно разложить по правилу произведения вероятностей:

$$P(text) = P(w_{\{0\}}, w_{\{1\}}, \dots, w_{\{n\}}) = P(w_0) \cdot P(w_1 | w_0) \cdot P(w_2 | w_1, w_0) \cdot \dots \cdot P(w_n | w_{\{n-1\}}, \dots, w_0)$$

RNN позволяют эффективно аппроксимировать эти условные вероятности, используя скрытое состояние h_t для компактного представления всей информации о префиксе $x_{0:t-1}$.

Механизм рекуррентного шага

На каждом временном шаге i сеть принимает на вход текущий элемент последовательности x_i и предыдущее скрытое состояние h_i , вычисляя новое состояние по формуле:

$$h_{\{i+1\}} = \sigma(W_{\{hid\}} \cdot h_i + W_{\{inp\}} \cdot x_i + b)$$

Для предсказания следующего элемента последовательности (например, слова x_{i+1}) используется слой **softmax** над скрытым состоянием:

$$P(x_{\{i+1\}} | x_{\{0:i\}}) = \text{softmax}(W_{\{out\}} \cdot h_{\{i+1\}} + b_{\{out\}})$$

Архитектуры для генерации

Специфические архитектуры RNN, предназначенные для генеративных задач:

1. **One-to-many (Decoder):** Принимает на вход один вектор состояния (например, признаковое описание изображения от CNN) и генерирует последовательность токенов (Image Captioning).
2. **Many-to-many (Seq2seq / Encoder-Decoder):** Самая распространенная архитектура для перевода или диалоговых систем. Сначала **Encoder** считывает входную последовательность в «вектор мысли» (thought vector), а затем **Decoder** разворачивает этот вектор в новую последовательность.

Трудности обучения

Обучение генеративных RNN методом обратного распространения ошибки во времени (BPTT) сопряжено с проблемами **затухающих и взрывающихся градиентов**. Для решения этих проблем используются:

- Архитектуры с пропусками связей (**LSTM, GRU**), которые позволяют градиентам течь через время почти без затухания.
- **Gradient Clipping** — принудительное ограничение нормы градиента для предотвращения расходимости.

2. Применение RL к дообучению моделей генерации последовательностей

Стандартное обучение моделей генерации (SFT — Supervised Fine-Tuning) методом максимизации правдоподобия (Maximum Likelihood Estimation) имеет ряд существенных ограничений, которые решаются с помощью методов обучения с подкреплением.

Проблема сдвига распределения (Exposure Bias)

При обучении с учителем модель на каждом шаге получает «идеальный» предыдущий токен из обучающей выборки ($y_t \sim \text{reference}$). Однако во время генерации (inference) модель вынуждена опираться на свои собственные, возможно ошибочные, предсказания ($y_t \sim \text{model}$). Малейшая ошибка на раннем этапе приводит к выходу из распределения данных, и сеть начинает генерировать «мусор». RL решает эту проблему, так как в этом режиме модель обучается непосредственно на своих собственных выводах.

Самокритичное обучение (Self-Critical Sequence Training, SCST)

SCST — это подход в рамках метода градиента стратегии (REINFORCE), который использует режим инференса как базовую линию (baseline) для снижения дисперсии.

- Агент (модель) генерирует две последовательности: одну путем сэмплирования ($a \sim \text{sampling}$), другую — жадно ($a_{\text{inference}} \sim \text{max}$).
- Награда вычисляется для обеих последовательностей по внешней метрике (например, CIDEr для описания картинок или BLEU для перевода).
- Формула преимущества (Advantage), используемая в градиенте:
$$A(s, a) = R(s, a) - R(s, a_{\text{inference}})$$
- Если сэмплированная последовательность оказалась лучше жадной, вероятность её генерации увеличивается.

RLHF и InstructGPT (Метод обучения на обратной связи от людей)

Современные большие языковые модели (LLM), такие как ChatGPT, проходят через трехэтапный процесс дообучения:

1. **SFT**: Дообучение на качественных примерах «инструкция — ответ». Одной только этой стадии недостаточно, так как она провоцирует галлюцинации: модель учится имитировать ответ, даже если не знает фактов.

2. **Reward Model (RM):** Обучение нейросети-критика, которая предсказывает оценку ответа человеком на основе ранжирования нескольких вариантов. Функция потерь RM минимизирует логарифм сигмоиды разности наград для «лучшего» (y_w) и «худшего» (y_l) ответов:

$$loss(\theta) = - \frac{1}{\binom{K}{2}} E_{\{(x, y_w, y_l) \sim D\}} [\log (\sigma(r_{\theta(x, y_w)} - r_{\theta(x, y_l)}))]$$

3. **RL Fine-Tuning (PPO):** Модель генерации (Actor) оптимизируется против RM с использованием алгоритма PPO, чтобы давать более полезные и безопасные ответы.

Direct Preference Optimization (DPO)

Более современная альтернатива RLHF, которая позволяет избежать обучения отдельной модели награды. DPO выводит оптимальную стратегию в закрытой форме и оптимизирует её напрямую через данные о предпочтениях пользователей, что вычислительно проще и стабильнее. Формула лосса DPO основывается на логарифмическом отношении вероятностей текущей и референсной моделей: $p * (y_1 > y_2 | x) = 1 / (1 + \exp(\beta \log(\pi * (y_2 | x) / \pi_{ref}(y_2 | x)) - \beta \log(\pi * (y_1 | x) / \pi_{ref}(y_1 | x))))$